

Personal Wealth Management System — SpecKit SDD POC

1. POC Objective

Build a small web-based **Personal Wealth Management System** to evaluate **SpecKit-based Spec-Driven Development (SDD)** end-to-end.

The system allows a user to:

- Create an investment portfolio
- Add investment holdings
- Record buy/sell transactions
- View portfolio value
- View basic portfolio performance

The POC should be intentionally small and should be implementable within approximately **one working day**.

The primary objective is to evaluate how effectively SpecKit can transform specifications into:

Specification → Design/Plan → Implementation → Tests

and how well the resulting application supports incremental specification changes.

2. Project Description

The Personal Wealth Management System is a simple application for tracking a user's investments.

A user can:

1. Create a portfolio
2. Add investment holdings
3. Record investment transactions
4. View current portfolio holdings
5. View portfolio summary and basic performance

The application uses manually entered investment prices.

No real-time market data or external brokerage integration is required.

3. Technology Stack

Frontend

- React
- TypeScript
- Vite
- Plain CSS

Backend

- Python
- FastAPI
- SQLAlchemy

Database

- SQLite

Testing

- Pytest
- React Testing Library

The stack is intentionally simple to minimize implementation and token overhead.

4. Functional Workflow

```
text id="4r5e0m" User      |      v Create Portfolio    |      v Add Investment      |      v
Record Transaction        |      v Update Holding      |      v Portfolio Dashboard  |
+-----+-----+         |      |                      |      v                      |
v Holdings                 |      | Performance         |      |                      |
```

5. Feature 1 — Create Portfolio

Requirement

A user can create a personal investment portfolio.

The user provides:

- Portfolio Name
- Base Currency

Example:

Portfolio Name: My Investments

Currency: INR

The system generates a unique portfolioId.

Example:

PORT-10001

After successful creation, the UI displays:

Portfolio created successfully.

Portfolio ID: PORT-10001

The frontend stores the portfolio ID and uses it for subsequent operations.

API

text id="8a4w9q" POST /api/portfolios

Request

```
json id="n7i6o8" {  "name": "My Investments",  "currency": "INR" }
```

Response

```
json id="0qdr5f" {  "portfolioId": "PORT-10001",  "name": "My Investments",  "currency": "INR" }
```

6. Feature 2 — Add Investment Holding

Requirement

A user can add an investment to the portfolio.

Supported investment types:

- STOCK
- MUTUAL_FUND
- ETF

The user provides:

- Investment Name
- Symbol
- Investment Type

Example:

Investment Name: ABC Bank
 Symbol: ABCBANK
 Type: STOCK

The system generates a unique holdingId.

Example:

HOLD-20001

API

text id="1i2x6v" POST /api/portfolios/{portfolioId}/holdings

Request

```
json id="8zhy9t" {  "name": "ABC Bank",  "symbol": "ABCBANK",  "type": "STOCK" }
```

Response

```
json id="6g7j6d" {  "holdingId": "HOLD-20001",  "portfolioId": "PORT-10001",  "name": "ABC Bank",  "symbol": "ABCBANK",  "type": "STOCK" }
```

7. Feature 3 — Record Investment Transaction

Requirement

The user can record a BUY or SELL transaction for an existing holding.

Transaction information:

- Transaction Type
- Quantity
- Price per Unit
- Transaction Date

Supported transaction types:

text id="d9e9oa" BUY SELL

Example:

Type: BUY
Quantity: 10
Price: ₹500
Date: 2026-09-01

The system calculates:

text id="l2w4p8" Transaction Value = Quantity × Price

Example:

10 × ₹500 = ₹5,000

API

text id="4v27a4" POST /api/holdings/{holdingId}/transactions

Request

```
json id="z8q4ft" {  "type": "BUY",  "quantity": 10,  "price": 500,  "transactionDate": "2026-09-01" }
```

Response

```
json id="eh9aqm" {  "transactionId": "TXN-30001",  "holdingId": "HOLD-20001",  "type": "BUY",  "quantity": 10,  "price": 500,  "value": 5000 }
```

8. Feature 4 — Portfolio Holdings

Requirement

The user can view all holdings in a portfolio.

For each holding, display:

- Investment Name
- Symbol
- Type
- Quantity
- Average Purchase Price
- Current Price
- Current Value

For this POC, the **current price is manually maintained** rather than retrieved from a market-data provider.

API

text id="9d2g1e" GET /api/portfolios/{portfolioId}/holdings

Example Response

```
json id="9o7s2r" [ {    "holdingId": "HOLD-20001",    "name": "ABC Bank",  
"symbol": "ABCBANK",    "quantity": 10,    "averagePrice": 500,  
"currentPrice": 550,    "currentValue": 5500  } ]
```

9. Feature 5 — Update Current Price

Requirement

The user can update the manually maintained current price for an investment.

This avoids integration with external market-data APIs while still allowing portfolio value and performance calculations.

API

text id="xw6a3f" PUT /api/holdings/{holdingId}/price

Request

```
json id="y7qv7x" {    "currentPrice": 550 }
```

Response

```
json id="rj3g7p" {    "holdingId": "HOLD-20001",    "currentPrice": 550 }
```

10. Feature 6 — Portfolio Summary

Requirement

The user can view an overall portfolio summary.

Display:

- Total Invested Amount
- Current Portfolio Value
- Profit/Loss
- Profit/Loss Percentage

Calculations:

```text id="y8f42g" Invested Amount = Sum of BUY transaction values - Sum of SELL transaction values

Current Value = Sum of holding quantity × current price

Profit/Loss = Current Value - Net Invested Amount

Profit/Loss % = Profit/Loss / Net Invested Amount × 100

## API

```text id="8m3k2y"  
GET /api/portfolios/{portfolioId}/summary

Example Response

json id="4x0x7n" { "portfolioId": "PORT-10001", "totalInvested": 5000,
"currentValue": 5500, "profitLoss": 500, "profitLossPercentage": 10 }

11. API Summary

The POC contains **6 APIs**:

| # | Method | Endpoint | Purpose |
|---|--------|--|------------------------|
| 1 | POST | /api/portfolios | Create portfolio |
| 2 | POST | /api/portfolios/{portfolioId}/holdings | Add holding |
| 3 | POST | /api/holdings/{holdingId}/transactions | Record BUY/SELL |
| 4 | GET | /api/portfolios/{portfolioId}/holdings | View holdings |
| 5 | PUT | /api/holdings/{holdingId}/price | Update current price |
| 6 | GET | /api/portfolios/{portfolioId}/summary | View portfolio summary |

This is enough to demonstrate a complete investment tracking workflow without creating a large application.

12. UI Scope

Use four simple screens.

Screen 1 — Portfolio Creation

```
```text id="zq9k2g" Personal Wealth Manager
```

Create Portfolio

Portfolio Name [\_\_\_\_\_]

Currency [ INR ▼ ]

[ Create Portfolio ]

After creation:

```
```text id="z5h6bd"
```

Portfolio Created

Portfolio ID: PORT-10001

[Open Portfolio]

13. Screen 2 — Add Investment

```
```text id="g7sl0c" My Investments
```

Add Investment

Investment Name [\_\_\_\_\_]

Symbol [\_\_\_\_\_]

Type [ STOCK ▼ ]

[ Add Investment ]

---

# 14. Screen 3 – Record Transaction



```text id="z3h4fn"  
ABC Bank (ABCBANK)

Current Quantity: 10

Transaction Type
[BUY ▼]

Quantity
[_____]

Price
[_____]

Date
[_____]

[Record Transaction]

15. Screen 4 — Portfolio Dashboard

```text id="n7p2w0" My Investments

Portfolio Value ₹55,000

Invested ₹50,000

Profit / Loss ₹5,000 (+10%)

---

Holdings

ABC Bank ABCBANK Quantity: 100 Avg Price: ₹500 Current Price: ₹550 Value: ₹55,000

---

[ Add Investment ] [ Record Transaction ] [ Update Price ]

Keep the dashboard simple. No advanced charts are required for the initial POC.

---

## # 16. Data Model

Use only three primary entities.

### ## Portfolio

```
```text id="8m9i8k"
portfolioId
name
currency
createdAt
```

Holding

```
text id="3d2r9a" holdingId portfolioId name symbol type currentPrice
createdAt
```

Transaction

```
text id="f3s1oe" transactionId holdingId type quantity price transactionDate
createdAt
```

Relationship:

```
text id="zv6j2x" Portfolio      |      | 1:N      v Holding      |      | 1:N      v
Transaction
```

17. Business Rules

Keep the business rules deliberately limited.

Portfolio

- Portfolio name is mandatory
- Currency is mandatory

Holding

- Investment name is mandatory
- Symbol is mandatory
- Investment type must be supported
- Holding must belong to an existing portfolio

Transaction

- Transaction type must be BUY or SELL
- Quantity must be greater than zero

- Price must be greater than zero
- Holding must exist
- SELL quantity cannot exceed the currently held quantity

Price

- Current price must be greater than zero

Portfolio Calculation

- Portfolio value is calculated from current holding quantities and current prices
- Profit/loss is calculated from current value and net invested amount

18. Transaction Rules

The core calculation is:

```
```text id="3r6gq1" BUY 10 @ ₹500 ↓ Quantity = 10 Invested = ₹5,000
```

```
BUY 5 @ ₹600 ↓ Quantity = 15 Invested = ₹8,000
```

```
SELL 5 @ ₹700 ↓ Quantity = 10
```

For the POC, the system does not need sophisticated tax, FIFO/LIFO, capital-gains, or tax-lot calculations.

---

## # 19. Error Handling

Use simple HTTP errors:

```
```text id="f0f1kv"
400 Bad Request
404 Not Found
409 Conflict
```

Example:

```
json id="g8i6ll" {  "error": "Sell quantity exceeds available holding
quantity" }
```

20. Repository Structure

```
text id="f715fu" personal-wealth/ | | frontend/ | | src/ | | | |
components/ | | | pages/ | | | services/ | | | App.tsx |
└─ package.json | | backend/ | | app/ | | | api/ | | main.py |
models/ | | | schemas/ | | | services/ | | | |
└─ tests/ | | requirements.txt | | README.md
```

21. Explicitly Out of Scope

Do not implement:

- Real-time stock prices
- NSE/BSE integration
- Brokerage integration
- Bank account integration
- Mutual fund provider integration
- Authentication/SSO
- Tax calculation
- Capital-gains tax
- Dividend tracking
- SIP automation
- Financial advice/recommendations
- Portfolio optimization
- AI investment recommendations
- Notifications
- Cloud deployment
- Microservices
- Advanced charts
- Multi-user access control

The objective is to evaluate **SpecKit SDD**, not to build a complete wealth-management product.

22. SpecKit SDD Execution Strategy

The project should be developed using a SpecKit-oriented SDD workflow.

The initial feature should move through:

text id="3n2o4d"	Feature Description	↓	Specification	↓	
Clarification / Requirements	↓	Plan	↓	Implementation	↓
Tests					

The specification should remain the primary reference for implementation.

23. Incremental Change Experiment

After the initial implementation is working, introduce a small change:

Change Request: Display the percentage allocation of each holding in the portfolio dashboard.

For example:

text id="17g6j9"	ABC Bank	₹55,000	70%	XYZ Fund	₹24,000	30%
------------------	----------	---------	-----	----------	---------	-----

The change requires updates to:

- Specification
- Data/calculation logic
- Backend API response
- Frontend dashboard
- Tests

This provides a simple way to evaluate SpecKit's ability to propagate a specification change through the implementation.

24. Optional Second Change

If time permits, introduce:

Change Request: Prevent a user from selling more units than currently held and display a clear validation message in the UI.

This tests whether the specification, backend validation, UI validation, and tests remain consistent.

25. Token Consumption Measurement

Measure Speckit usage across each major stage.

Stage	Input Tokens	Output Tokens	Total
Specification			
Clarification			
Planning			
Implementation			
Testing			
Change Request			
Total			

Also record:

- Number of interactions
 - Number of generated artifacts
 - Context size
 - Time per stage
 - Number of implementation iterations
 - Manual corrections required
-

26. POC Success Criteria

Functional

- User can create a portfolio
- Portfolio ID is generated
- User can add investments
- User can record BUY/SELL transactions
- Holdings are calculated correctly
- Current price can be updated
- Portfolio value is calculated correctly
- Profit/loss is calculated correctly
- User can view the portfolio dashboard
- Invalid transactions are rejected

SDD

- Specification clearly describes the feature

- Implementation follows the specification
- Tests cover the important requirements
- Specification changes can be translated into code changes
- Existing functionality continues to work after changes

Efficiency

- POC can be completed in approximately one day
 - Token consumption can be measured by stage
 - The project remains small enough for repeatable SDD experiments
-

27. Final POC Boundary

1 application

React + Vite + TypeScript + CSS

FastAPI + SQLAlchemy

SQLite

3 database entities

6 APIs

4 UI screens

3 core investment concepts: Portfolio, Holding, Transaction

1 portfolio calculation model

1–2 incremental changes

No external integrations

The purpose of this POC is to establish a **clean SpecKit SDD baseline**, similar in size and complexity to the Banking KYC BMAD POC, so that the two approaches can later be compared on:

- SDD artifact quality
- Code quality
- Requirement-to-code traceability
- Incremental change handling
- Token consumption
- Implementation effort

- Developer intervention